

Webprogramozás alapjai

Online Napló

Webprogramozás Projekt Dokumentáció

GitHub repository: <https://github.com/kovzsolti/OnlineNaplo>

Online alkalmazás: <https://onlinenaplo.onrender.com>

Elevator pitch video: https://gdfhu-my.sharepoint.com/:v:/g/personal/s525tw_neptun_gde_hu/IQDrSHsJzJLlSY9Vz_1ZONe_AbRGFogAQUFpcPrJ0j7gAfo?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=SbKRd4

Név: Kovács Zsolt

Neptunkód: S525TW

Távoktatás

Tartalomjegyzék

Webprogramozás alapjai	1
Online Napló	1
<i>Webprogramozás Projekt Dokumentáció</i>	1
Projekt bemutatása	3
Használt technológiák	3
Projekt struktúra	4
Az alkalmazás működése	4
Frontend bemutatása	5
Backend bemutatása	5
Adatbázis.....	6
REST API végpontok	6
Tesztelés.....	6
Kiegészítő funkciók	7
Verziókezelés	7
Online elérhetőség	7
Elevator Pitch videó	7
Összefoglalás.....	8

Projekt bemutatása

A célom egy egyszerű full-stack webalkalmazás elkészítése volt frontend, backend és relációs adatbázis használatával. Az alkalmazás egy online napló rendszer, amely lehetőséget biztosít jegyzetek létrehozására, megjelenítésére, szerkesztésére és törlésére.

A fejlesztést Visual Studio Code-ban oldottam meg.

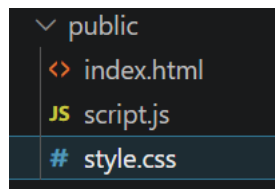
A projekt során REST API alapú kommunikáció került megvalósításra a frontend és backend között.

Az alkalmazást elérhetővé tettem online Render környezetben és böngészőn keresztül futtatható. Ez egyfajta CI/CA megoldást biztosít.

Használt technológiák

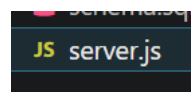
Frontend

- HTML
- CSS
- JavaScript



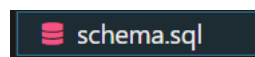
Backend

- Node.js
- Express.js



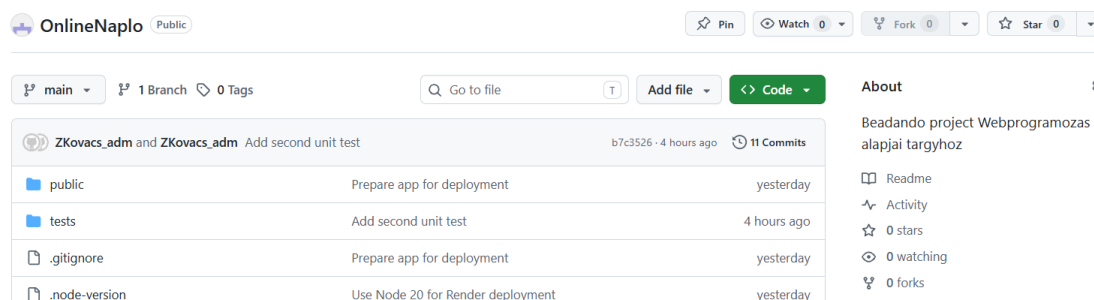
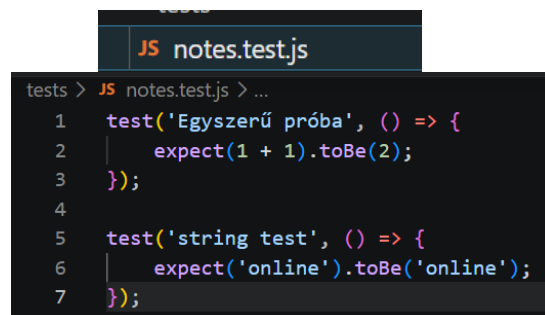
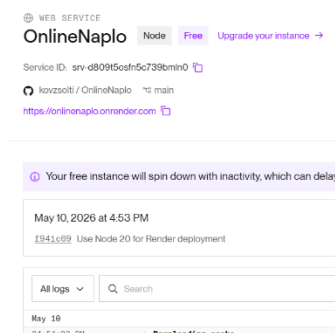
Adatbázis

- SQLite

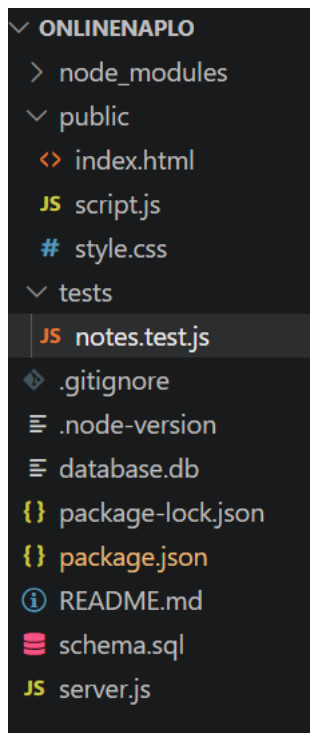


Egyéb technológiák

- Git
- GitHub
- Render
- Jest



Projekt struktúra



FrontEnd

BackEnd

Relációs Adatbázis

Tesztek

Az alkalmazás működése

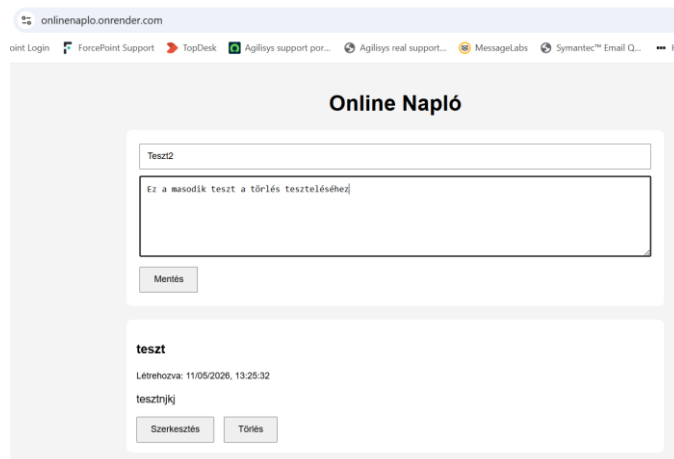
Az alkalmazás lehetőséget biztosít jegyzetek kezelésére, webes környezetben. CRUD (Create, Retrieve, Update, Delete) megvalósítás.

Az én konkrét beadandó feladatomban ezek az alábbiak.

- új jegyzetet hozhat létre,
- megtekintheti a korábban létrehozott jegyzeteket,
- módosíthatja a meglévő jegyzeteket,
- valamint törölheti azokat.

Az adatok SQLite relációs adatbázisban kerülnek tárolásra.

A frontend JavaScript segítségével REST API hívásokat küld a backend számára.



Frontend bemutatása

A frontend HTML-t, CSS-t és JavaScript-et használ.

index.html

Az index.html fájl tartalmazza az alkalmazás alap felhasználói felületét. Ilyenek, például:

- jegyzet cím mező,
- jegyzet tartalom mező,
- mentés gomb,
- jegyzetek listázása.

style.css

A style.css fájlban beállítottam egy pár egyszerű formázást a felhasználóbarát interfacehez, megjelenéshez.

script.js

A script.js fájl kezeli az API fetcheket, és a funkciókat.

- a REST API hívásokat,
- a jegyzetek betöltését,
- új jegyzetek létrehozását,
- szerkesztést,
- törlést,
- valamint a frontend logikát.

Backend bemutatása

A backendhez Node.js és Express.js használtam.

A backend feladata:

- REST API végpontok biztosítása,
- adatbázis kapcsolat kezelése,

```
// GET all notes
app.get('/api/notes', (req, res) => {
  db.all('SELECT * FROM notes ORDER BY created_at DESC', [], (err, rows) => {
    if (err) {
      return res.status(500).json({ error: err.message });
    }
    res.json(rows);
  });
});

// POST new note
app.post('/api/notes', (req, res) => {
  const { title, content } = req.body;

  db.run(
    'INSERT INTO notes (title, content) VALUES (?, ?)',
    [title, content],
    function (err) {
      if (err) {
        return res.status(500).json({ error: err.message });
      }
      res.json({
        title,
        content,
        created_at: this.date
      });
    }
  );
});
```

- CRUD műveletek végrehajtása.

A server.js fájl tartalma.

- az Express szerver konfigurációja,
- az SQLite kapcsolat,
- az API végpontok,
- valamint az alkalmazás indítása.

```
const db = new sqlite3.Database('./database.db', (err) => {
  if (err) {
    console.error('Database connection error:', err.message);
  } else {
    console.log('Connected to SQLite database.');
```

Adatbázis

Az alkalmazás SQLite relációs adatbázist használ. A notes tábla mezői az alábbi táblázatban láthatóak.

Mező	Típus	Leírás
id	INTEGER	Egyedi azonosító
title	TEXT	Jegyzet címe
content	TEXT	Jegyzet tartalma
created_at	DATETIME	Létrehozás időpontja

Az adatbázis séma a schema.sql fájlban található.

```
schema.sql
1 CREATE TABLE IF NOT EXISTS notes (
2   id INTEGER PRIMARY KEY AUTOINCREMENT,
3   title TEXT NOT NULL,
4   content TEXT NOT NULL,
5   created_at DATETIME DEFAULT CURRENT_TIMESTAMP
6 );
```

REST API végpontok

Metódus	Végpont	Funkció
GET	/api/notes	Jegyzetek lekérdezése
POST	/api/notes	Új jegyzet létrehozása
PUT	/api/notes/:id	Jegyzet módosítása
DELETE	/api/notes/:id	Jegyzet törlése

Tesztelés

A beadandó tartalmaz két alap tesztet Jest használatával.

A tesztek futtatása “npm test”-el működik.

A tesztfuttatás eredménye: Test Suites: 1 passed, 1 total, Tests: 2 passed, 2 total

```
PS C:\Temp\WebProg\OnlineNaplo> npm test

> onlinenaplo@1.0.0 test
> jest

PASS tests/notes.test.js
  ✓ Egyszerű próba (1 ms)
  ✓ string test (1 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.381 s, estimated 1 s
Ran all test suites.
PS C:\Temp\WebProg\OnlineNaplo> git add .
```

A rendszer manuális tesztelése során ellenőrzésre kerültek az alap funkciók.

- új jegyzet létrehozása,
- jegyzetek megjelenítése,
- szerkesztés,
- törlés,
- valamint az online működés Render környezetben.

Kiegészítő funkciók

Az applikáció a kötelező funkciókon felül az alábbi kiegészítő elemeket tartalmazza:

Cloud szolgáltatás használata

Az alkalmazás Render cloud környezetben került publikálásra, így online, böngészőből közvetlenül kipróbálható.

CI/CD integráció

A projekt GitHub repositoryhoz kapcsolódik, és a Render a GitHubon lévő forráskódból végzi az alkalmazás felépítését, tesztelését és deployolását. Így a GitHubra feltöltött változtatások alapján új deployment indítható.

Verziókezelés

A project verziókezelésére Git-et használtam.

A source code GitHub repositoryban került tárolásra.

GitHub repository: <https://github.com/kovzsolti/OnlineNaplo>

Online elérhetőség

Az alkalmazás Render környezetben került publikálásra.

Online alkalmazás: <https://onlinenaplo.onrender.com>

Elevator Pitch videó

A videó az alábbi linken érhető el:

https://gdfhu-my.sharepoint.com/:v:/g/personal/s525tw_neptun_gde_hu/IQDrSHsJzJLISY9Vz_1ZONe_AbRGFogAQUFpcPrJ0j7gAfo?nav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIoiJPbmVEcmI2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=SbKRd4

Összefoglalás

A projekt során sikeresen elkészítettem egy működő full-stack webalkalmazás frontend, backend és relációs adatbázis használatával.

Az alkalmazás CRUD műveleteket, REST API kommunikációt használ, valamint online környezetben is elérhető.

A projekt segítségével gyakorlati tapasztalatot sikerült szereznem az alábbi területeken:

- webprogramozás,
- REST API fejlesztés,
- adatbázis kezelés,
- frontend-backend kommunikáció,
- valamint online (Render) deployment.